

Node.js

What is Node.js?

- Node.js is a JavaScript runtime built on Google's **V8 JavaScript engine**
(The same V8 engine also powers Chrome)
 - It is a platform to develop **server-side** and **networking applications** in JavaScript
 - Developed by **Ryan Dahl in 2009**
 - It is **free and open-source**
 - It can run on various operating systems like **macOS, Linux, and Windows**
-

Node.js Components

- **JavaScript Runtime:** V8 (other engines can be used too)
 - **Event Loop:** Powered by **libuv**
 - **Standard Library:**
 - Filesystem Access
 - HTTP
 - OS
 - etc
-

Common Misconceptions

1. Node.js is single-threaded

- Not all parts of Node.js are single-threaded
- **Event Loop** and **V8** run in a single thread. It has single main call stack.
- Your code has single threaded. If you run very expensive application then you **block event loop**

2. Node.js uses thread pool for all I/O

- Threads need to block while we do the IO so Node does not use **Thread Pool for IO**
- Node uses thread pool:
 - when there is no OS async API that allows it do the non-blocking API
 - For **cryptographic operations**
 - For **DNS lookups**

3. The Event Loop runs in one thread and V8 runs in separate thread

Node.js Architecture

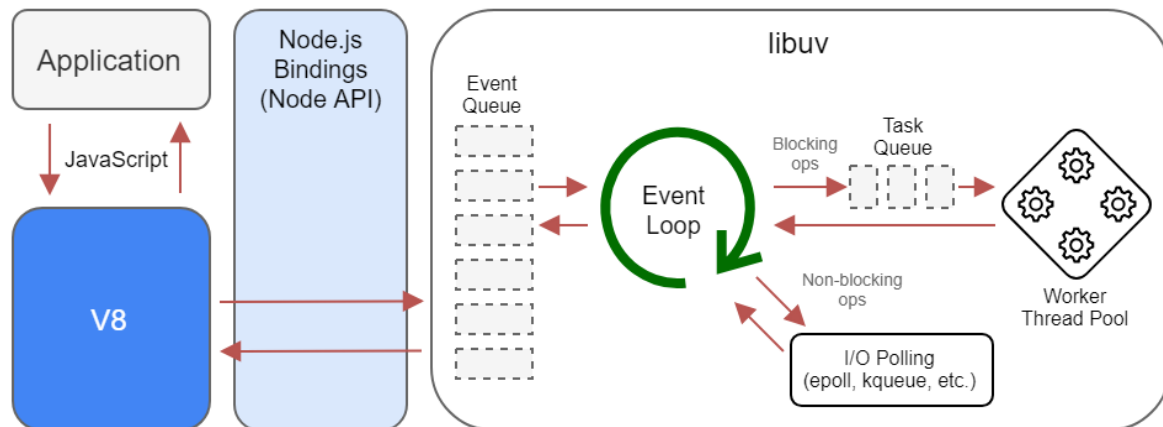


Figure: Node.js Internal Architecture with Event Loop and libuv

Node.js Features

- It uses event-driven, non-blocking I/O model.
- It has a single threaded but highly scalable architecture.
- It is a lightweight and efficient.
- It has its own package ecosystem which has thousands of packages for different purpose.
- It never buffers the data. Rather it simply outputs the data in chunks.

Node.js Use Cases

☑ Where to Use Node.js

- **I/O bound applications**
- **Data streaming applications**
- **IoT applications**
- **JSON API-based applications**
- **Single Page Applications (SPA)**

✗ Where Not to Use Node.js

- **CPU-intensive applications**
- **Heavy server-side processing**

Advantages of Node.js

- Single programming language for both client and server

- Easily scalable
 - Large community support
 - High performance
 - Reduced response time
 - Faster development time
 - Freedom to develop apps your way
 - **Full-stack JavaScript Application**
-

Built in objects

- console
 - used to access the console / terminal
 - `log()`: used to print output on console
-

Module

- used to reuse the code
- module is any file with an extension: `.js`, `.jsx`, `.mjs`
- node module contains ONLY the JS code
- node adds an object called as a module to every module (JS file)
- exporting entities (function/variable/constant/class/interface) from module
 - `module.exports = {...}`
- importing a module
 - `require()`
 - `require` function returns the exported property of imported module